

CS4340 – Tutorial 1S (Assessed)

Theme

- Understanding software systems and software development lifecycle
- Understanding class diagram and how it informs software system design

*This tutorial will be assessed. Please submit your solutions electronically via Blackboard at the end of the tutorial session. Your submission should be a PDF file named after your name and the name of the tutorial i.e., **FirstName_Surname_1S.pdf**.*

Assessed Exercises

1. Assume you work for a company called CompanyName that wants to develop an **assistive system for elderly people with limited mobility**:

The system will be comprised of a **smart doorbell, a WiFi access point, and a robot** who can maneuver indoor and open the main door. It will be set up by a technician, who will also install all the hardware and a WiFi network. The robot will have access to the rest of the items and, to go beyond GDPR requirements. The system will not access the Internet.

The only way that the system can communicate with the outside is through a phone-based alarm system to warn health services. The alarm will be activated if the user asks for it or if the robot finds that the user has lost consciousness (except for when they are sleeping). In addition to controlling the alarm, the robot will open the main door to visitors who are granted access by the user when they knock on the door or trigger the doorbell. The process of allowing in visitors is as follows:

- i the robot detects the knock with its microphone or receives the doorbell signal,
- ii the robot approaches the user while streaming the doorbell camera's view to show the visitor,
- iii the robot asks the user whether the the visitor should be allowed in depending on user's verbal reply,
- iv the robot opens the door or asks the visitor to leave, based on the user's choice.

Known visitors who are allowed in will have to connect to the local WiFi access point and request the door to be open using a web service integrated in the WiFi access point in the future. They will be registered through a wired connection to the access point, by contacting the technician. Each known visitor will have their own password, which they can update when authenticated in the system web service.

- a) Identify FOUR actors of the proposed system. (Actor = someone or something that interacts with the system but is not a component of the system.)

(2 marks)

- b) The proposed system could have many software components, depending on one's choice of design. Identify THREE software components that could be a part of this system and depict them using a UML component diagram (2 marks per correctly drawn component). There should be 2–3 connections between these components' interfaces and sockets (1 mark deduction per missing connection or incorrect description).

Reminder: Describe interfaces and sockets. Each interface/socket description should refer to the capabilities/needs of its component only, i.e., not referring specifically to other components, so that it is clear how the component could be (re-)used in other systems. Hardware components should not be considered, but the software controlling them can.

(6 marks)

- c) Identify TWO specific potential important properties or behaviours of this system that are clearly *emergent*. Avoid generic properties such as "reliable". Instead, be specific to this system.

(2 marks)

2. You are tasked with designing an **AstonAI server system** that offers **GPU-based cloud computing to train models**, each represented by objects interacting with one another. Below is a structure of the system represented in Java code snippets (and Python equivalence). Your goal is to analyze the code and draw a **Class Diagram** that interprets all the classes and relationships for the AstonAI's developers.

Identify all classes of the system (3 marks), show attributes and methods (2 marks), and supply appropriate relationships and multiplicity between classes (5 marks).

```
import java.util.List;

class WebUI {
    private String url;
    private String session;

    public WebUI(String url, String session) {
        this.url = url;
        this.session = session;
    }

    public void display() {
        System.out.println("Displaying web interface at " + url);
    }
}

class AppUI {
    private String version;
    private String session;

    public AppUI(String version, String session) {
        this.version = version;
        this.session = session;
    }

    public void launch() {
        System.out.println("Launching app UI version " + version);
    }
}

class GPU {
    private int memoryCapacity;

    public GPU(int memoryCapacity) {
        this.memoryCapacity = memoryCapacity;
    }

    public void runComputation() {
        System.out.println("RAM: " + memoryCapacity + " GB memory");
    }
}

class Handler {
    private String requestType;

    public Handler(String requestType) {
        this.requestType = requestType;
    }
}
```

```

    }

    public void handleRequest(List<GPU> gpus) {
        for (GPU gpu : gpus) {
            gpu.runComputation();
        }
    }
}

class AstonAIServer {
    private List<GPU> gpus;
    private WebUI webUI;
    private AppUI appUI;
    private Handler handler;

    public AstonAIServer(List<GPU> gpus, WebUI webUI,
        Handler handler, AppUI appUI)
    {
        this.gpus = gpus;
        this.webUI = webUI;
        this.handler = handler;
        this.appUI = appUI;
    }

    public AstonAIServer(List<GPU> gpus, WebUI webUI, Handler handler) {
        this.gpus = gpus;
        this.webUI = webUI;
        this.handler = handler;
        this.appUI = null;
    }

    public void startServer() {
        webUI.display();
        if (appUI != null) {
            appUI.launch();
        }
        handler.handleRequest(gpus);
    }
}

```

Below are the Python snippets.

```

1  class WebUI:
2      def __init__(self, url, session):
3          self._url = url
4          self._session = session
5
6      def display(self):
7          print(f"Displaying web interface at {self._url}")
8
9  class AppUI:
10     def __init__(self, version, session):
11         self._version = version
12         self._session = session
13
14     def launch(self):
15         print(f"Launching app UI version {self._version}")
16
17 class GPU:
18     def __init__(self, memory_capacity):
19         self._memory_capacity = memory_capacity
20
21     def run_computation(self):
22         print(f"Running computation with {self._memory_capacity} GB memory")
23
24 class Handler:
25     def __init__(self, request_type):
26         self._request_type = request_type
27
28     def handle_request(self, gpus):
29         print(f"Handling {self._request_type} request using multiple GPUs")
30         for gpu in gpus:
31             gpu.run_computation()
32
33 class AstonAIServer:
34     def __init__(self, gpus, web_ui, handler, app_ui=None):
35         self._gpus = gpus
36         self._web_ui = web_ui
37         self._app_ui = app_ui
38         self._handler = handler
39
40     def start_server(self):
41         self._web_ui.display()
42         if self._app_ui:
43             self._app_ui.launch()
44         self._handler.handle_request(self._gpus)

```
